>>>>>>>>>>>> 제어 구조의 설계 원리를 중심으로 배우는 >>>>>>>>>>>>

프로그래밍의 정석 파이썬

도경구 지음

생능출판

CHAPTER 4

재귀와 반복 : 자연수 계산

자연수
Natural Number

$$N = \{0, 1, 2, 3, \ldots\}$$

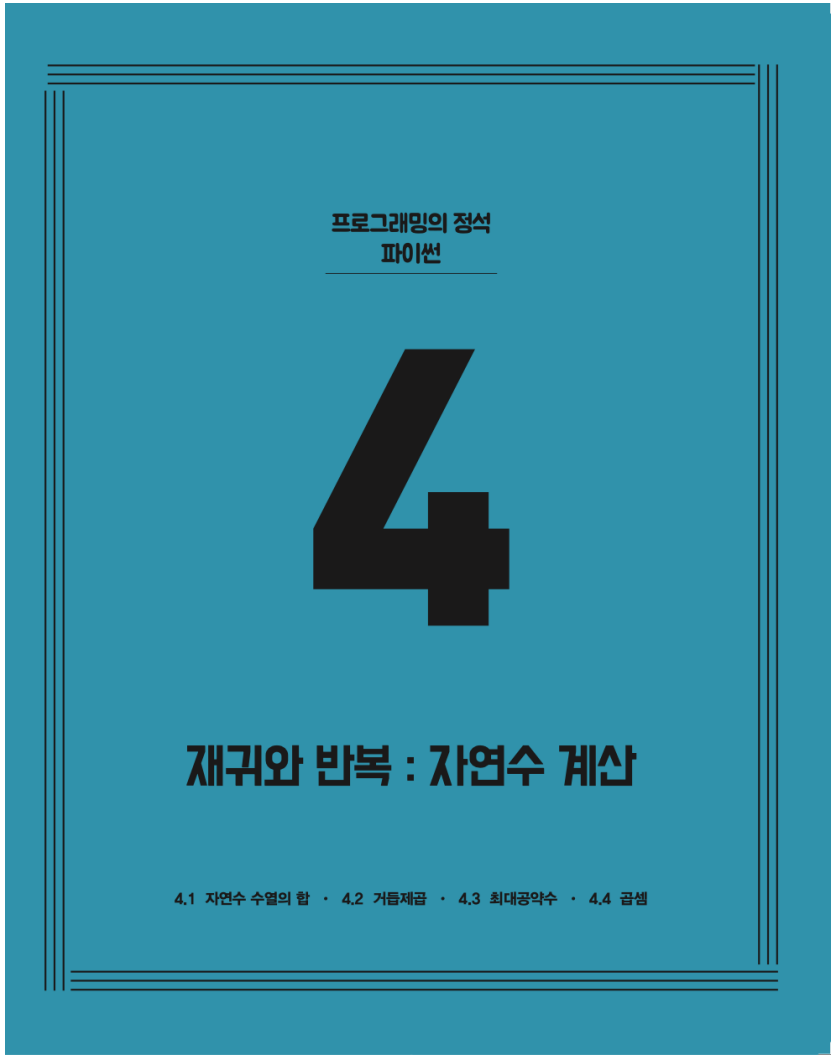
자연수
Natural Number

수학적 귀납歸納,인덕

Mathematical Induction

(1)	기초 Basis	0은 자연수 이다.
(2)	귀납 Induction	n이 자연수이면, n+1도 자연수이다.
(3)	그 외에 다른 자연수는 없다.	

자연수는 무한이 많이 있지만
이 귀납 정의를 이용하면
아무리 큰 자연수라도
유한한 절차를 거쳐서
자연수임을 확인할 수 있다.



CHAPTER 4

재귀와 반복 : 자연수 계산

✔ 4.1 자연수 수열의 합

4.2 거듭제곱

4.3 최대공약수

4.4 곱셈

자연수 수열의 합

덧셈만으로 0부터 n까지 누적 합을 계산하여 내주는 함수 sigma(n)을 만들자.

0 + 1 + 2 + 3 + … + n

귀납

(1)	기초 Basis	0은 자연수 이다.
(2)	인덕 Induction	n이 자연수이면, n+1도 자연수이다.

$$\text{sigma}(0) = 0 \quad [base]$$

$$\text{sigma}(n+1) = \text{sigma}(n) + (n+1) \quad [induction]$$

재귀식recursive equation

순진무구 알고리즘

재귀 함수recursive function

$$\text{sigma}(n) = \begin{cases} \text{sigma}(n-1) + n & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

반복조건

종료조건

재귀 호출recursive call

$$\text{sigma}(n) = \begin{cases} \text{sigma}(n-1) + n & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

다시쓰기rewriting

$$\begin{aligned}
\text{sigma}(5) &= \text{sigma}(4) + 5 && [1] \\
&= (\text{sigma}(3) + 4) + 5 && [2] \\
&= ((\text{sigma}(2) + 3) + 4) + 51 && [3] \\
&= (((\text{sigma}(1) + 2) + 3) + 4) + 5 && [4] \\
&= ((((\text{sigma}(0) + 1) + 2) + 3) + 4) + 5 && [5] \\
&= ((((0 + 1) + 2) + 3) + 4) + 5 && [6] \\
&= (((1 + 2) + 3) + 4) + 5 \\
&= ((3 + 3) + 4) + 5 \\
&= (6 + 4) + 5 \\
&= 10 + 5 \\
&= 15
\end{aligned}$$

$$\text{sigma}(n) = \begin{cases} \text{sigma}(n-1) + n & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

code : 4-1.py

```python
def sigma(n):
    if n > 0:
        return sigma(n-1) + n
    else:
        return 0
```

```
def sigma(n):
    if n > 0:
return sigma(n-1) + n
      else:
       return 0
```

```
sigma(5)
  =>
```

재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **sigma 함수의 계산 복잡도**
  - 시간
    - 덧셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 인수가 n 일 때 덧셈을 총 n번(= 재귀 호출을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 재귀 함수를 호출하는 횟수에 비례 (답을 구해온 뒤에 더해야 할 수를 기억해 둘 공간 필요)
    - 인수가 n 일 때 재귀 호출을 총 n번 하므로 필요 공간은 n에 비례

꼬리재귀 함수

Tail Recursive Function

재귀

code : 4-1.py

```
def sigma(n):
    if n > 0:
        return sigma(n-1) + n
    else:
        return 0
```

꼬리 재귀

code : 4-2.py

```
def sigma(n):
    return loop(n, 0)

def loop(n, total):
    if n > 0:
        return loop(n-1, n+total)
    else:
        return total
```

```
def sigma(n):
 return loop(n,0)

def loop(n,total):
      if n > 0:
 return loop(n-1,n+total)
        else:
       return total
```

```
sigma(5)
 =>
```

꼬리재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **꼬리재귀 sigma 함수의 계산 복잡도**
  - 시간
    - 덧셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 인수가 n 일 때 덧셈을 총 n번(= 재귀 호출을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 인수의 크기와 상관없이 일정

code : 4-2.py

```
1  def sigma(n):
2      return loop(n, 0)
3
4  def loop(n, total):
5      if n > 0:
6          return loop(n-1, n+total)
7      else:
8          return total
```

보조 함수

지역화

code : 4-3.py

```
1  def sigma(n):
2      def loop(n,total):
3          if n > 0:
4              return loop(n-1,n+total)
5          else:
6              return total
7      return loop(n,0)
```

지역 함수

local function

캡슐화

Encapsulation

재귀

code : 4-1.py

```
def sigma(n):
    if n > 0:
        return sigma(n-1) + n
    else:
        return 0
```

하향식
Top-down

꼬리 재귀

code : 4-3.py

```
def sigma(n):
    def loop(n,total):
        if n > 0:
            return loop(n-1,n+total)
        else:
            return total
    return loop(n,0)
```

상향식
Bottom-up

꼬리 재귀

code : 4-3.py

```
def sigma(n):
    def loop(n,total):
        if n > 0:
            return loop(n-1,n+total)
        else:
            return total
    return loop(n,0)
```

while 루프

code : 4-4.py

```
def sigma(n):
    n, total = n, 0
    while n > 0:
        n, total = n-1, n+total
    return total
```

code : 4-3.py

꼬리 재귀

```
def sigma(n):
    def loop(n,total):
        if n > 0:
            return loop(n-1,n+total)
        else:
            return total
    return loop(n,0)
```

code : 4-5.py

while 루프

```
def sigma(n):
    total = 0
    while n > 0:
        n, total = n-1, n+total
    return total
```

계산 비용 분석

- 시간
- 공간

while 루프 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **while 루프로 작성한 sigma의 계산 복잡도**
  - 시간
    - 덧셈의 횟수 (= 루프를 반복하는 횟수)에 비례
    - 인수가 n 일 때 덧셈을 총 n번(= 루프 반복을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 인수의 크기와 상관없이 일정

지정문의 실행 순서

code : 4-5.py

```
1  def sigma(n):
2      total = 0
3      while n > 0:
4          n, total = n-1, n+total
5      return total
```

```
n = n - 1
total = n + total
```

```
total = n + total
n = n - 1
```

정리

- 하향식으로 작동하는 재귀 함수는 실행 논리를 직관적으로 표현할 수 있어서 코딩하기 쉬운 반면, 공간 효율이 떨어질 수 있다.
- 상향식으로 작동하는 꼬리재귀 함수나 `while` 루프 버전로 변환하면 공간을 절약할 수 있다.
- 재귀 함수, 꼬리재귀 함수, while 문은 모두 구조적으로 연관이 있어서 상호 기계적으로 변환할 수 있다.
- 상대적으로 사고하기 쉬운 하향식으로 재귀 함수를 먼저 작성하고, 꼬리재귀와 `while` 루프로 차례로 변환하여 함수를 다듬는 것은 권장할만한 코딩 습관이다.

파이썬과 재귀의 궁합

가우스 알고리즘

$$\text{sigma}(n) = 1 + 2 + \cdots + (n - 1) + n$$

수학적 귀납법 검증

- 자연수 n을 중심으로 펼치는 주장 또는 명제 P(n)은 모든 n에 대해서 성립하는가?

- <u>수학적 귀납법 증명</u>
  - 기초 단계 base step:

    P(0)은 참이다.
  - 귀납 단계 induction step:

    임의의 자연수 n에 대해서, P(n)이 참이면, P(n+1)도 참이다.

가우스 알고리즘의 검증

정리 4.1.1 임의의 자연수 n까지의 누적 합 sigma(n)은 다음 식으로 계산한다.

$$\text{sigma}(n) = \frac{n \times (n + 1)}{2}$$

증명(수학적 귀납법)

>>>>>>>>>>> 제어 구조의 설계 원리를 중심으로 배우는 >>>>>>>>>>>

프로그래밍의 정석 파이썬

도경구 지음

생능출판

pp.166~167

실습 4.1 구간 수열의 합

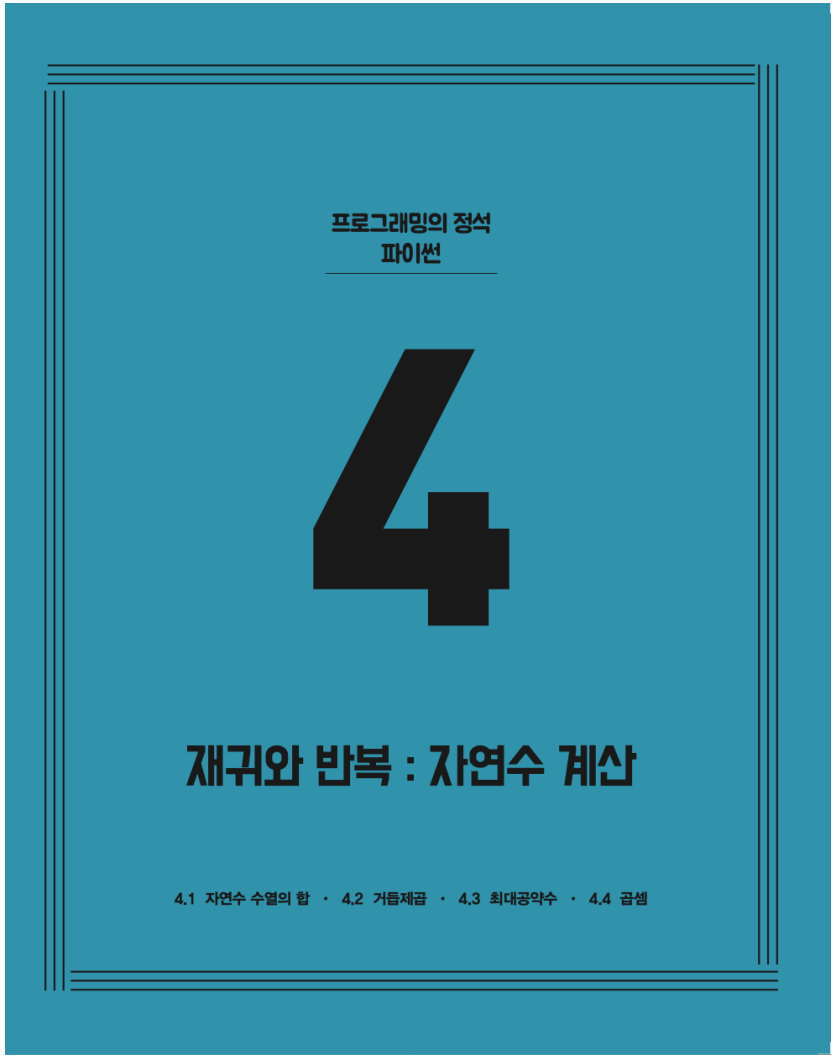
$$sumrange(m,n) = \begin{cases} m + sumarage(m+1,n) & \text{if } m \leq n \\ 0 & \text{if } m > n \end{cases}$$

```python
def sumrange(m,n):
    if m <= n:
        return m + sumrange(m+1, n)
    else:
        return 0

# Test Code
print(sumrange(3, 2)) # 0
print(sumrange(3, 3)) # 3
print(sumrange(3, 4)) # 7
```

```
def sumarage(m, n):
    def loop(m, total):
        if m <= n:
            return loop(m+1, ______________)
        else:
            return _____________
    return loop(m, _____________)
```

```
def sumarage(m, n):
    total = _____
    while m <= n:
        total = __________
        m = __________
    return total
```



CHAPTER 4

재귀와 반복 : 자연수 계산

4.1 자연수 수열의 합

✔ 4.2 거듭제곱

4.3 최대공약수

4.4 곱셈

거듭제곱 b^n 계산하기

```
b**n
```

```
pow(b,n)
```

거듭제곱 b^n 계산하기

bn**

pow(b,n)

b의 n 거듭제곱인 b^n을 계산하여 내주는 함수
power(b,n)을 별도로 만들어보자.
b는 정수, n은 자연수로 제한하고,
음수 인수는 모두 0으로 취급하기로 한다.

순진무구 알고리즘

$$b^n = \begin{cases} b \times b^{n-1} & \text{if } n > 0 \\ 1 & \text{if } n \leq 0 \end{cases}$$

순진무구 알고리즘

$$b^n = \begin{cases} b \times b^{n-1} & \text{if } n > 0 \\ 1 & \text{if } n \leq 0 \end{cases}$$

code : 4-13.py

```
def power(b,n):
    if n > 0:
        return b * power(b,n-1)
    else:
        return 1
```

```
def power(b,n):
    if n > 0:
return b * power(b,n-1)
    else:
     return 1
```

```
power(2,5)
  =>
```

재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **power 함수의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 인수가 n 일 때 곱셈을 총 n번(= 재귀 호출을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 재귀 함수를 호출하는 횟수에 비례 (답을 구해온 뒤에 곱해야 할 수를 기억해 둘 공간 필요)
    - 인수가 n 일 때 재귀 호출을 총 n번 하므로 필요 공간은 n에 비례

재귀

code : 4-13.py

```
1 def power(b,n):
2     if n > 0:
3         return b * power(b,n-1)
4     else:
5         return 1
```

재귀

code : 4-13.py

```
def power(b,n):
    if n > 0:
        return b * power(b,n-1)
    else:
        return 1
```

꼬리 재귀

code : 4-14.py

```
def power(b,n):
    def loop(b,n,prod):
        if n > 0:
            return loop(b,n-1,b*prod)
        else:
            return prod
    return loop(b,n,1)
```

재귀

code : 4-13.py

```
def power(b,n):
    if n > 0:
        return b * power(b,n-1)
    else:
        return 1
```

꼬리 재귀

code : 4-15.py

```
def power(b,n):
    def loop(n,prod):
        if n > 0:
            return loop(n-1,b*prod)
        else:
            return prod
    return loop(n,1)
```

```
def power(b,n):
 def loop(n,prod):
      if n > 0:
  return loop(n-1,b*prod)
       else:
      return prod
 return loop(n,1)
```

```
power(2,5)
   =>
```

꼬리재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **꼬리재귀 power 함수의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 인수가 n 일 때 곱셈을 총 n번(= 재귀 호출을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 인수의 크기와 상관없이 일정

code : 4-15.py

꼬리 재귀

```
def power(b,n):
    def loop(n,prod):
        if n > 0:
            return loop(n-1,b*prod)
        else:
            return prod
    return loop(n,1)
```

code : 4-16.py

while 루프

```
def power(b,n):
    prod = 1
    while n > 0:
        prod = b * prod
        n = n - 1
    return prod
```

계산 비용 분석

- 시간
- 공간

while 루프 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **while 루프로 작성한 power의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 루프를 반복하는 횟수)에 비례
    - 인수가 n 일 때 곱셈을 총 n번(= 루프 반복을 총 n번) 하므로 계산시간은 n에 비례
  - 공간
    - 인수의 크기와 상관없이 일정

계산 시간 비용 절약?

$$b^n = (b^2)^{\frac{n}{2}} \qquad \text{if } n > 0 \; and \; even(n)$$

n

$1 + n/2$

곱셈의 횟수

나눠 풀기 알고리즘

Divide-and-Conquer

$$b^n = \begin{cases} (b \times b)^{\frac{n}{2}} & \text{if } n > 0 \text{ and } n \bmod 2 = 0 \\ b \times b^{n-1} & \text{if } n > 0 \text{ and } n \bmod 2 \neq 0 \\ 1 & \text{if } n \leq 0 \end{cases}$$

나눠 풀기 알고리즘

Divide-and-Conquer

$$b^n = \begin{cases} (b \times b)^{\frac{n}{2}} & \text{if } n > 0 \text{ and } n \bmod 2 = 0 \\ b \times b^{n-1} & \text{if } n > 0 \text{ and } n \bmod 2 \neq 0 \\ 1 & \text{if } n \leq 0 \end{cases}$$

code : 4-17.py

```python
def power(b,n):
    if n > 0:
        if n % 2 == 0:
            return power(b*b,n//2)
        else:
            return b * power(b,n-1)
    else:
        return 1
```

```
def power(b,n):
    if n > 0:
    if n % 2 == 0:
    return power(b*b,n//2)
        else:
    return b * power(b,n-1)
        else:
        return 1
```

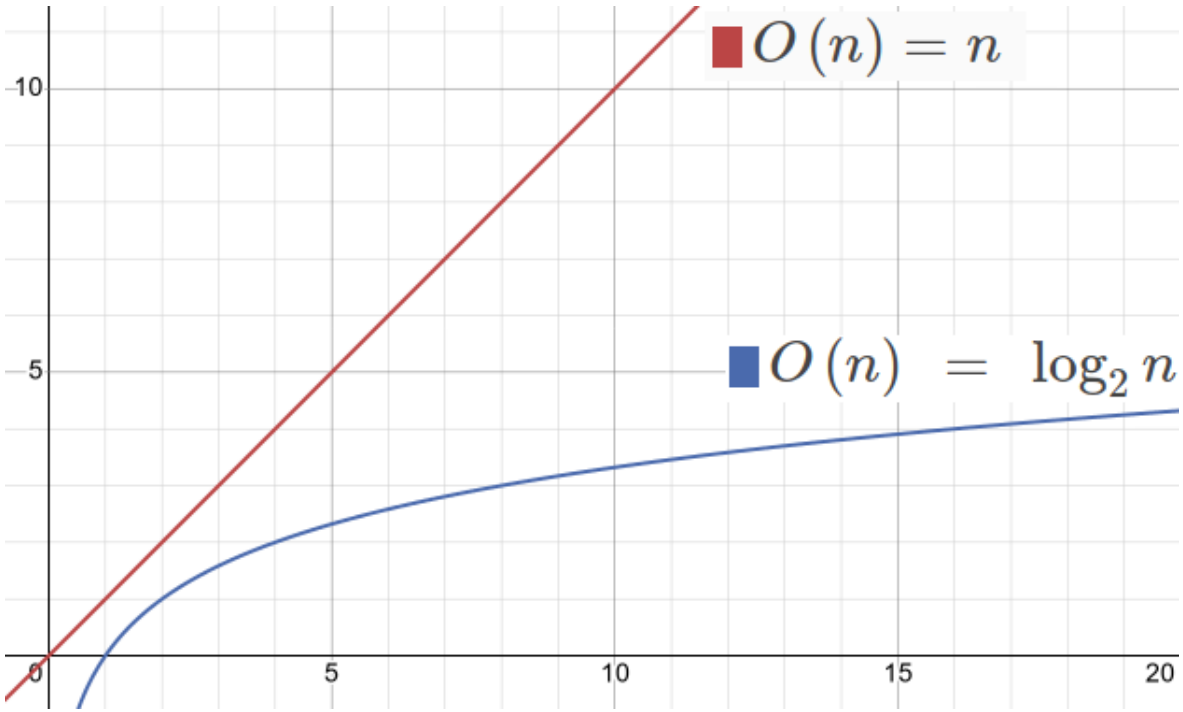
```
power(2,7)
=>
```

재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **power 함수의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 둘째 인수 n이 짝수 일 때 인수의 크기가 반으로 작아지므로 호출 횟수를 대략 따져 보면 약 $\log_2 n$ 번이 되므로 계산시간은 $\log_2 n$에 비례함
  - 공간
    - 재귀 함수를 호출하는 횟수에 비례 (답을 구해온 뒤에 곱해야 할 수를 기억해둘 공간 필요)
    - 둘째 인수 n이 짝수 일 때 위와 마찬가지로 재귀 호출을 약 $\log_2 n$ 번 하므로 계산시간은 $\log_2 n$에 비례함

n vs. $\log_2 n$

n	$\log_2 n$
2	1
4	2
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1024	10
2048	11
4096	12
8192	13
16384	14
32768	15
65536	16



power(2, 66506)	66506							
	33253	33252						
		16626						
		8313	8312					
			4156					
			2078					
			1039	1038				
				519	518			
					259	258		
						129	128	
							64	
							32	
							16	
							8	
							4	
							2	
							1	0

재귀

code : 4-17.py

```
def power(b,n):
    if n > 0:
        if n % 2 == 0:
            return power(b*b,n//2)
        else:
            return b * power(b,n-1)
    else:
        return 1
```

꼬리 재귀

code : 4-18.py

```
def power(b,n):
    def loop(b,n,prod):
        if n > 0:
            if n % 2 == 0:
                return loop(b*b,n//2,prod)
            else:
                return loop(b,n-1,b*prod)
        else:
            return prod
    return loop(b,n,1)
```

```
def power(b,n):
def loop(b,n,prod):
        if n > 0:
        if n % 2 == 0:
    return loop(b*b,n//2,prod)
            else:
    return loop(b,n-1,b*prod)
            else:
        return prod
return loop(b,n,1)
```

```
power(2,7)
    =>
```

꼬리재귀 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **꼬리재귀 power 함수의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 재귀 함수를 호출하는 횟수)에 비례
    - 둘째 인수 n이 짝수 일 때 인수의 크기가 반으로 작아지므로 호출 횟수를 대략 따져보면 약 $\log_2 n$ 번이 되므로 계산시간은 $\log_2 n$에 비례함
  - 공간
    - 인수의 크기와 상관없이 일정

꼬리 재귀

code : 4-18.py

```
def power(b,n):
    def loop(b,n,prod):
        if n > 0:
            if n % 2 == 0:
                return loop(b*b,n//2,prod)
            else:
                return loop(b,n-1,b*prod)
        else:
            return prod
    return loop(b,n,1)
```

while 루프

code : 4-19.py

```
def power(b,n):
    prod = 1
    while n > 0:
        if n % 2 == 0:
            b = b * b
            n = n // 2
        else:
            n = n - 1
            prod = b * prod
    return prod
```

계산 비용 분석

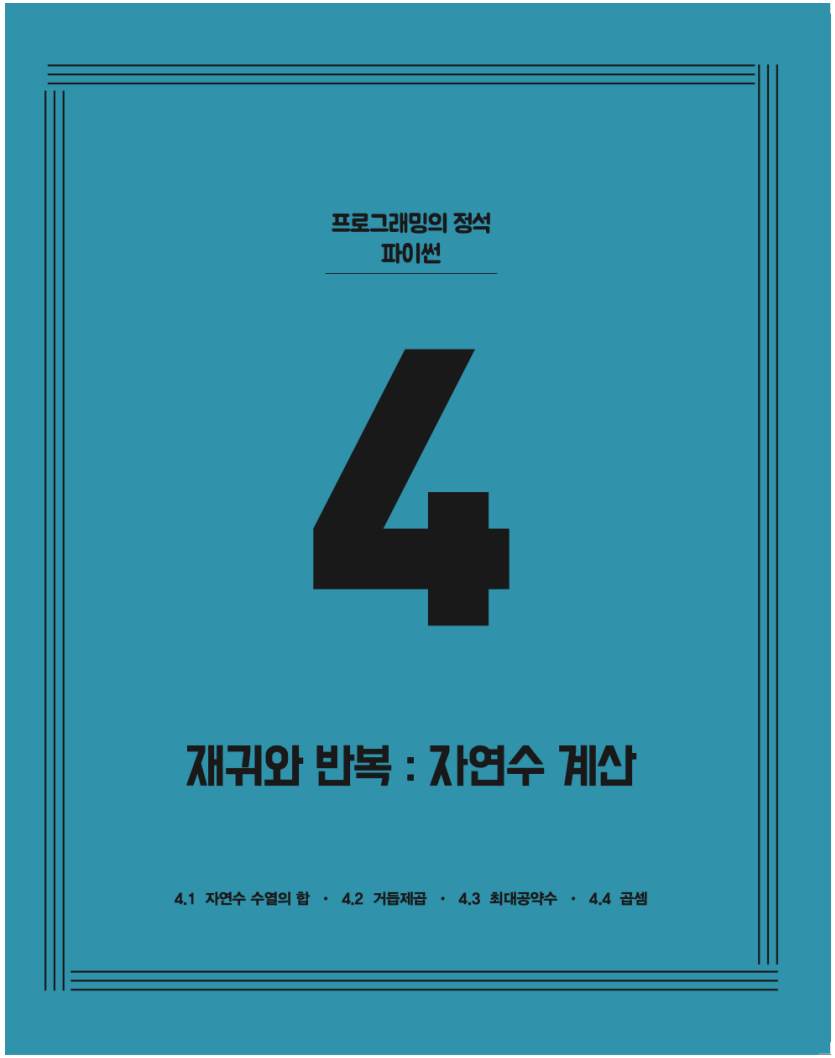
- 시간
- 공간

while 루프 함수의 계산 비용 분석

- **계산복잡도**computational complexity
  - 시간 : 프로그램이 얼마나 빨리 답을 계산하는가?
  - 공간 : 답을 계산하면서 얼마나 많은 공간을 사용하는가?
- **while 루프로 작성한 power의 계산 복잡도**
  - 시간
    - 곱셈의 횟수 (= 루프를 반복하는 횟수)에 비례
    - 둘째 인수 n이 짝수 일 때 인수의 크기가 반으로 작아지므로 계산 시간은 $\log_2 n$에 비례함
  - 공간
    - 인수의 크기와 상관없이 일정

code : 4-19.py

```
1  def power(b,n):
2      prod = 1
3      while n > 0:
4          if n % 2 == 0:
5              b = b * b
6              n = n // 2
7          else:
8              n = n - 1
9              prod = b * prod
10     return prod
```



CHAPTER 4

재귀와 반복 : 자연수 계산

4.1 자연수 수열의 합

4.2 거듭제곱

✔ 4.3 최대공약수

4.4 곱셈

약수
Divisor

정수의 약수는
나누어서
나머지 없이 떨어지는
양수를 말한다.

예를 들어,
42의 약수는
42, 21, 14, 7, 6, 3, 2, 1

공약수
Common Divisor

두 정수의 공약수는
두 정수의 약수 중에서
공통되는 수를 말한다.

예를 들어,
54의 약수는 54, 27, 18, 9, 6, 3, 2, 1
24의 약수는 24, 12, 8, 6, 4, 3, 2, 1
54와 24의 공약수는
6, 3, 2, 1

최대공약수
Greatest Common Divisor

공약수들 중에서
가장 큰 수를
최대공약수라고 한다.

예를 들어,
54와 24의 공약수
6, 3, 2, 1
중에서
최대공약수는 6이다.

```
from math import gcd
```

최대공약수 계산 함수

두 자연수 m과 n을 인수로 받아
두 자연수의 최대공약수를 계산하는
함수 gcd을 만들자.

유클리드 알고리즘

Euclid

- 나눗셈을 이용한 알고리즘
- 두 수의 최대공약수는 두 수의 차이로도 나누어지는 성질을 이용

$$\gcd(m, n) = \begin{cases} \gcd(n, m \bmod n) & \text{if } n \neq 0 \\ m & \text{if } n = 0 \end{cases}$$

유클리드 알고리즘

Euclid

- 나눗셈을 이용한 알고리즘
- 두 수의 최대공약수는 두 수의 차이로도 나누어지는 성질을 이용

$$\gcd(m,\ n) = \begin{cases} \gcd(n,\ m \bmod n) & \text{if } n \neq 0 \\ m & \text{if } n = 0 \end{cases}$$

code : 4-20.py

```
def gcd(m,n):
    if n != 0:
        return gcd(n,m%n)
    else:
        return m
```

유클리드 알고리즘

Euclid

```
def gcd(m,n):
    if n != 0:
        return gcd(n,m%n)
    else:
        return m
```

```
  gcd(18,48)
=>
```

```
def gcd(m, n):
    while n != 0:
        pass # Fill your code in this space

    return m
```

나눠 풀기 알고리즘

Divide-and-Conquer

- 뺄셈, 2로 곱하기, 2로 나누기 연산만 이용한 알고리즘

$$
\gcd(m, n) = \begin{cases}
2 \times \gcd(\frac{m}{2}, \frac{n}{2}) & \text{if even}(m) \text{ and even}(n) \\
\gcd(\frac{m}{2}, n) & \text{if even}(m) \text{ and odd}(n) \\
\gcd(m, \frac{n}{2}) & \text{if odd}(m) \text{ and even}(n) \\
\gcd(m, \frac{n-m}{2}) & \text{if odd}(m) \text{ and odd}(n) \text{ and } m \leq n \\
\gcd(n, \frac{m-n}{2}) & \text{if odd}(m) \text{ and odd}(n) \text{ and } m > n \\
n & \text{if } m = 0 \\
m & \text{if } n = 0
\end{cases}
$$

$$\gcd(m,n) = \begin{cases} 2 \times \gcd(\frac{m}{2}, \frac{n}{2}) & \text{if even}(m) \text{ and even}(n) \\ \gcd(\frac{m}{2}, n) & \text{if even}(m) \text{ and odd}(n) \\ \gcd(m, \frac{n}{2}) & \text{if odd}(m) \text{ and even}(n) \\ \gcd(m, \frac{n-m}{2}) & \text{if odd}(m) \text{ and odd}(n) \text{ and } m \leq n \\ \gcd(n, \frac{m-n}{2}) & \text{if odd}(m) \text{ and odd}(n) \text{ and } m > n \\ n & \text{if } m = 0 \\ m & \text{if } n = 0 \end{cases}$$

```python
def even(n):
    return n % 2 == 0

def odd(n):
    return n % 2 == 1
```

```python
def gcd(m,n):
    if not (m == 0 or n == 0):
        if even(m) and even(n):
            return 2 * gcd(m//2,n//2)
        elif even(m) and odd(n):
            return gcd(m//2,n)
        elif odd(m) and even(n):
            return gcd(m,n//2)
        elif m <= n:
            return gcd(m,(n-m)//2)
        else:
            return gcd(n,(m-n)//2)
    else:
        if m == 0:
            return n
        else:
            return m
```

나눠 풀기 알고리즘

Divide-and-Conquer

code : 4-23.py

```
def gcd(m,n):
    if not (m == 0 or n == 0):
        if even(m) and even(n):
            return 2 * gcd(m//2,n//2)
        elif even(m) and odd(n):
            return gcd(m//2,n)
        elif odd(m) and even(n):
            return gcd(m,n//2)
        elif m <= n:
            return gcd(m,(n-m)//2)
        else:
            return gcd(n,(m-n)//2)
    else:
        if m == 0:
            return n
        else:
            return m
```

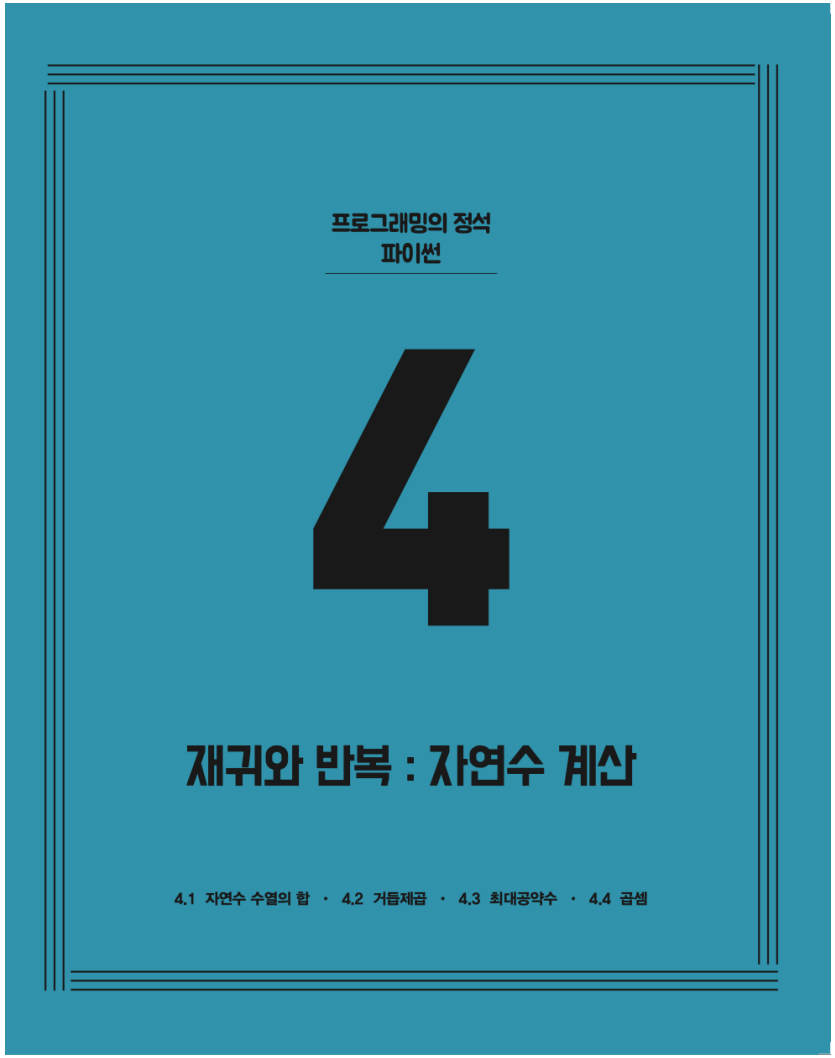
gcd(18,48)

=>

```
def gcd(m, n):
    def loop(m, n, k):
        if not (m == 0 or n == 0):
            if even(m) and even(n):
                return loop(m//2, n//2, _______)
            elif even(m) and odd(n):
                return loop(m//2, n, k)
            elif odd(m) and even(n):
                return loop(m, n//2, k)
            elif m <= n:
                return loop(m, (n-m)//2, k)
            else:
                return loop(m, (n-m)//2, k)
        else:
            if m == 0:
                return ________
            else:
                return ________

    return loop(m, n, 1)
```

```
def gcd(m, n):
    k = 1
    while not (m == 0 or n == 0):
        if even(m) and even(n):
            m, n, k = __________
        elif even(m) and odd(n):
            m = __________
        elif odd(m) and even(n):
            n = __________
        elif m <= n:
            n = (n-m)//2
        else:
            m, n = _________
    if m == 0:
        return __________
    else: # n == 0
        return __________
```



CHAPTER 4

재귀와 반복 : 자연수 계산

4.1 자연수 수열의 합

4.2 거듭제곱

4.3 최대공약수

✔ 4.4 곱셈

곱셈

덧셈/뺄셈 알고리즘

$$m \times n = \begin{cases} m + m \times (n - 1) & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

곱셈

덧셈/뺄셈 알고리즘

$$m \times n = \begin{cases} m + m \times (n-1) & \text{if } n > 0 \\ 0 & \text{if } n = 0 \end{cases}$$

```
def mult(m,n):
    if n > 0:
        return m + mult(m,n-1)
    else:
        return 0
```

```
1  def mult(m,n):
2      if n > 0:
3          return m + mult(m,n-1)
4      else:
5          return 0
```

```
   mult(3,6)
=>
```

>>>>>>>>>>> 제어 구조의 설계 원리를 중심으로 배우는 >>>>>>>>>>>

프로그래밍의 정석 파이썬

도경구 지음

생능출판

pp.185~188

실습 4.5 덧셈/뺄셈 알고리즘 : 꼬리재귀 함수 버전

실습 4.6 덧셈/뺄셈 알고리즘 : while 루프 버전

```
def mult(m, n):
    def loop(n, ans)
        if n > 0:
            return __________
        else:
            return __________
    return loop(n, _________)
```

```
def mult(m, n):
    ans = ____
    while _________:
        pass # Fill your code in this space

    return __________
```

곱셈

덧셈/뺄셈/절반 알고리즘

$$m \times n = (m + m) \times (n \div 2) \quad \text{if} \quad \text{even}(n)$$

곱셈

덧셈/뺄셈/절반 알고리즘

$$m \times n = (m + m) \times (n \div 2) \quad \text{if} \ \ \text{even}(n)$$

$$m \times n = \begin{cases} (m + m) \times (n \div 2) & \text{if } n > 0 \text{ and even}(n) \\ m + m \ \ \times (n - 1) & \text{if } n > 0 \text{ and odd}(n) \\ 0 & \text{if } n = 0 \end{cases}$$

>>>>>>>>>>> 제어 구조의 설계 원리를 중심으로 배우는 >>>>>>>>>>>

프로그래밍의 정석 파이썬

도경구 지음

생능출판

pp.188~191

실습 4.7 덧셈/뺄셈/절반 알고리즘 : 재귀 함수 버전

실습 4.8 덧셈/뺄셈/절반 알고리즘 : 꼬리재귀 함수 버전

실습 4.9 덧셈/뺄셈/절반 알고리즘 : while 루프 버전

```
def fastmult(m, n):
    if n > 0:
        if n % 2 == 0:
            return __________
        else:
            return __________
    else:
        return _________
```

```
def fastmult(m, n):
    def loop(m, n, ans):
        if n > 0:
            pass # Fill your code in this space

        else:
            return __________
    return loop(m, n, _________)
```

```
def fastmult(m, n):
    ans = ____
    while n > 0:
        pass # Fill your code in this space


    return ans
```

곱셈

러시아 농부 알고리즘

- 곱할 두 수를 나란히 적는다.
- 첫째 수는 두 배를 하고, 둘째 수는 절반을 하여(나머지는 버림) 다음 줄에 나란히 적는다.

57×86

- 이 과정을 둘째 수가 1이 될 때까지 계속한다.
- 둘째 수가 짝수인 줄은 모두 지운다.
- 남은 줄의 첫째 수를 모두 더한 값이 답이다.

>>>>>>>>>>> 제어 구조의 설계 원리를 중심으로 배우는 >>>>>>>>>>>

프로그래밍의 정석 파이썬

도경구 지음

pp.191~194

실습 4.10 러시아 농부 알고리즘 : 재귀 함수 버전

실습 4.11 러시아 농부 알고리즘 : 꼬리재귀 함수 버전

실습 4.12 러시아 농부 알고리즘 : while 루프 버전

```
def russian_mult(m, n):
    def loop(m, n)
        if n > 1:
            pass # Fill your code in this space

        else: # n == 1
            pass # Fill your code in this space

    if n > 0:
        return loop(m, n)
    else:
        return 0
```

```
def russian_mult(m, n):
    def loop(m, n, a)
        if n > 0:
            pass # Fill your code in this space

        else: # n == 1
            pass # Fill your code in this space

    if n > 0:
        return loop(m, n, ____)
    else:
        return 0
```

```
def russian_mult(m, n):
    if n > 0:
        pass # Fill your code in this space


    else:
        return 0
```